

LAPORAN PRAKTIKUM STRUKTUR DATA

Queue



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 27 April 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum **“Queue”** ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi dan pemahaman hasil kegiatan praktikum yang bertujuan untuk memahami bagaimana konsep dari Queue. Penulis juga terbuka untuk menerima saran dan kritik guna perbaikan di masa mendatang.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah basis data, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.

Padang, 27 April 2026

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM STRUKTUR DATA	i
KATA PENGANTAR.....	ii
DAFTAR PUSTAKA.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Landasan teori	3
2.2 Langkah Pengerjaan	3
BAB III KESIMPULAN.....	17
DAFTAR PUSTAKA.....	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data yang efisien dan terstruktur merupakan fondasi utama dalam pengembangan perangkat lunak modern. Seiring meningkatnya kompleksitas sistem, pengembang tidak hanya dituntut memahami cara menyimpan data, tetapi juga mampu memisahkan logika penggunaan dari detail implementasi. Konsep *Abstract Data Type* (ADT) hadir sebagai solusi melalui prinsip abstraksi yang mendefinisikan tipe data berdasarkan perilaku dan operasi yang diizinkan, bukan representasi memorinya. Dengan memisahkan antarmuka (*interface*) dari implementasi, ADT mendukung *information hiding*, enkapsulasi, serta membentuk sistem yang modular, *reusable*, dan mudah diuji.

Meskipun bersifat teoretis, pemahaman ADT memerlukan implementasi langsung untuk menguji kebenaran operasi, menangani kondisi batas (*boundary conditions*), dan memvalidasi kontrak spesifikasi yang ditetapkan. Penguasaan konsep ini juga menjadi prasyarat penting sebelum mempelajari algoritma lanjutan maupun arsitektur perangkat lunak berbasis komponen. Oleh karena itu, praktikum ini dilaksanakan untuk memberikan pengalaman langsung dalam merancang, mengimplementasikan, dan menguji ADT sesuai spesifikasi teknis.

1.2 Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Memahami struktur ADT Queue
2. Memahami langkah-langkah menulis ADT Queue

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Meningkatkan pemahaman tentang ADT Queue
2. Meningkatkan pemahaman bagaimana cara menulis ADT Queue

BAB II

PEMBAHASAN

2.1 Landasan teori

Struktur data adalah cara penyimpanan, pengorganisasian, dan pengaturan data di dalam media penyimpanan komputer sehingga data tersebut dapat digunakan secara efisien. Sedangkan data adalah representasi dari fakta dunia nyata. Fakta atau keterangan tentang kenyataan yang disimpan, direkam atau direpresentasikan dalam bentuk tulisan, suara, gambar, sinyal atau simbol.

Definisi ADT (Abstract Data Type) sendiri dalam struktur data merupakan sebuah spesifikasi atau kontrak yang mendefinisikan antarmuka suatu tipe data. ADT tidak menentukan bagaimana tipe data diimplementasikan, melainkan berfokus pada operasinya dan perilaku yang diharapkan dari tipe data tersebut.

ADT Queue merupakan struktur data abstrak yang mengatur kumpulan entitas dalam urutan tertentu, dimana elemen ditambahkan di satu ujung (rear/back) dan dihapus dari ujung lainnya. Queue mengikuti prinsip FIFO (First In First Out), artinya elemen yang pertama kali dimasukkan ke dalam queue adalah elemen yang akan pertama kali dikeluarkan.

2.2 Langkah Pengerjaan

- IterasiQueue_2511532017
 1. Buka eclipse dan buat package baru dengan nama pekan4_2511532017 dan class IterasiQueue_2511532017.
 2. Setelah dibuat, kita akan membuat program iterasi dengan ADT Queue

```
package pekan4_2511532017;
import java.util.LinkedList;
import java.util.Iterator;
import java.util.Queue;

public class IterasiQueue_2511532017 {

    public static void main(String[] args) {
```

3. Pertama kita inisialisasi Queue dengan variabel q yang bertipe Queue<String> yang berarti hanya menampung objek data berupa String, dengan objek yang dibuat berupa LinkedList.

```
q.add("Praktikum");
q.add("Struktur");
q.add("Data");
q.add("Dan");
q.add("Algoritma");
```

4. Method add() digunakan untuk menambahkan elemen ke ujung belakang(rear) queue. Dengan elemen pertama yang masuk (“Praktikum”) akan berada di depan dan (“Algoritma”) di paling belakang.

```
Iterator<String> iterator= q.iterator();
while (iterator.hasNext()) {
    System.out.print(iterator.next()+" ");
}
}
```

5. q.iterator() mengembalikan objek iterator yang akan menelusuri elemen-elemen di dalam q. iterator tidak akan menghapus elemen. Ia hanya akan berfungsi untuk membaca/traversing.

```
Iterator<String> iterator= q.iterator();
while (iterator.hasNext()) {
    System.out.print(iterator.next()+" ");
}
}
```

6. Pengulangan dimana jika masih ada elemen berikutnya, mengembalikan nilai true dan jika sudah habis maka false. iterator.next() akan mengambil nilai saat ini dan akan memajukan pointer iterator ke elemen berikutnya lalu dicetak.

- QueueArray_2511532017

1. Kita akan membuat class baru untuk QueueArray_2511532017

```
package pekan4_2511532017;

public class QueueArray_2511532017 {
    int front_2017, rear_2017, size_2017;
    int capacity_2017;
    int array_2017[];
```

2. Atribut yang digunakan berupa front_2017 yang merupakan indeks pertama(paling depan) yang akan di dequeue, rear_2017 indeks elemen terakhir tempat elemen akan di enqueue, size_2017 jumlah

elemen yang sedang tersimpan di queue, capacity_2017 kapasitas maksimum array dan array_2017[] yang merupakan array statis bertipe int sebagai wadah penyimpanan.

```
public QueueArray_2511532017 (int capacity_2017) {
    this.capacity_2017 = capacity_2017;
    front_2017 = this.size_2017=0;
    rear_2017 = capacity_2017-1;
    array_2017 = new int [this.capacity_2017];
}
```

3. Constructor yang akan mengatur kapasitas sesuai parameter front_2017 = this.size_2017 mengatur size jadi 0 lalu front juga jadi 0, rear_2017 = capacity_2017 – 1 dengan memulai rear di indeks terakhir operasi (rear + 1) % capacity pada enqueue pertama akan menghasilkan 0 dan new int[this.capacity_2017] akan mengalokasi memori array di heap.

```
boolean isFull_2511532017(QueueArray_2511532017 queue) {
    return (queue.size_2017 == queue.capacity_2017);
}
boolean isEmpty_2511532017(QueueArray_2511532017 queue) {
    return (queue.size_2017==0);
}
```

4. Method mengecek apakah penuh/kosong berdasarkan perbandingan size_2017 dengan kapasitas atau 0.

```
void enqueue_2511532017 (int item) {
    if (isFull_2511532017 (this))
        return;
    this.rear_2017=(this.rear_2017+1) % this.capacity_2017;
    this.array_2017[this.rear_2017] = item;
    this.size_2017= this.size_2017+1;
    System.out.println(item + " enqueue to queue");
}
```

5. Method untuk menambah elemen, jika penuh keluar tetapi jika kosong tambah dengan (rear + 1) % capacity yaitu memindahkan rear maju kedepan satu langkah, jika sudah diakhir array %capacity akan mengembalikan ke indeks 0, lalu simpan item di posisi rear baru dan tambah size_2017 dan print informasi cetak.

```
int dequeue_2511532017 () {
    if (isEmpty_2511532017 (this))
        return Integer.MIN_VALUE;
    int item = this.array_2017[this.front_2017];
    this.front_2017 = (this.front_2017+1) % this.capacity_2017;
    this.size_2017=this.size_2017-1;
    return item;
}
```

6. Method dequeue, cek kosong jika iya kembalikan Integer.MIN_VALUE sebagai penanda error, ambil elemen di front.

(front + 1) % capacity untuk menggeser penunjuk depan maju, kurangi size_2017 dan kembalikan elemen yang dihapus.

```
int front_2511532017() {
    if (isEmpty_2511532017 (this))
        return Integer.MIN_VALUE;

    return this.array_2017[this.front_2017];
}
int rear_2511532017() {
    if (isEmpty_2511532017(this))
        return Integer.MIN_VALUE;
    return this.array_2017[this.rear_2017];
}
```

7. Method front dan rear mirip dengan dequeue tetapi tidak menghapus elemen dan tidak mengubah size dia Cuma melihat data paling didepan atau paling belakang.

```
void display_2511532017 () {
    int i;
    if (front_2017 == rear_2017) {
        System.out.println("\nAntrian Kosong\n");
        return;
    }
    for (i=front_2017; i<rear_2017; i++) {
        System.out.printf(" %d <--", array_2017[i]);
    }
    return;
}
```

8. Method untuk display jika front_2017 sama dengan rear_2017 maka cetak kosong, lalu loop for (i=front_2017; i<rear_2017; i++) dan gagal saat rear< front.

- QueueArrayDriver_2511532017

1. kita akan membuat class baru untuk QueueArrayDriver_2511532017

```
package pekan4_2511532017;

public class QueueArrayDriver_2511532017 {

    public static void main(String[] args) {
        QueueArray_2511532017 queue = new QueueArray_2511532017(1000);
    }
}
```

2. new QueueArray_2511532017(1000) memanggil constructor untuk membuat objek antrian dengan kapasitas 1000 slot, dengan state awal front_2017 = 0, rear_2017 = 999, size_2017 = 0, array_2017 dialokasikan dengan 1000 elemen bernilai 0

```
queue.enqueue_2511532017(10);
queue.enqueue_2511532017(20);
queue.enqueue_2511532017(30);
queue.enqueue_2511532017(40);
```

3. Operasi enqueue, setiap pemanggilan menjalankan logika enqueue yang ada pada kode sebelumnya. Dengan 10 sebagai elemen pertama dan 40 sebagai elemen paling ujung.

```
System.out.println("item didepan "+ queue.front_2511532017());
System.out.println("item paling dibelakang "+ queue.rear_2511532017());
System.out.println("tampilan queue");
queue.display_2511532017();
```

4. Operasi peek front_2511532017() mengakses array_2017[0] (output 10) dan rear_2511532017() mengakses array_2017[3] (output 40).

```
queue.display_2511532017();
```

5. Memanggil method display yang akan mencetak berdasarkan kode display pada kode sebelumnya.

```
System.out.println();
System.out.println(queue.dequeue_2511532017()+" dihapus dari queue");
System.out.println("item didepan "+queue.front_2511532017() );
System.out.println("item dibelakang "+ queue.rear_2511532017());
System.out.println("tampilan queue setelah satu data dihapus");
queue.display_2511532017();
}
```

6. Operasi dequeue, dequeue_2511532017() mengambil nilai dari front (array[0] = 10), menggeser front ke 1, dan mengurangi size menjadi 3, lalu akan dicetak.

```
System.out.println();
System.out.println(queue.dequeue_2511532017()+" dihapus dari queue");
System.out.println("item didepan "+queue.front_2511532017() );
System.out.println("item dibelakang "+ queue.rear_2511532017());
System.out.println("tampilan queue setelah satu data dihapus");
queue.display_2511532017();
}
```

7. Mencetak elemen pertama dan terakhir setelah elemen pertama dihapus
8. Output

```

10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
item didepan 10
item paling dibelakang 40
tampilan queue
10 <-- 20 <-- 30 <-- 40 <--
10 dihapus dari queue
item didepan 20
item dibelakang 40
tampilan queue setelah satu data dihapus
20 <-- 30 <-- 40 <--

```

- QueueLinkedList_2511532017

1. Kita akan membuat class dengan nama QueueLinkedList_2511532017

```

package pekan4_2511532017;

import java.util.*;

public class QueueLinkedList_2511532017 {

    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();

```

2. Inisialisasi queue Queue<Integer> q = new LinkedList<>(); dimana q bertipe interface Queue<Integer> tetapi objek yang dibuat adalah LinkedList. LinkedList mengimplementasikan Queue dan Deque, struktur doubly linkedlist memungkinkan operasi add() dan remove().

```

        for (int i_2017 = 0; i_2017<6; i_2017++)
            q.add(i_2017);

```

3. Looping berjalan dari 0 hingga 5. q.add(i_2017) memasukkan elemen keujung belakang (rear) antrian. Dengan urutan masuk 0, 1, 2, 3, 4, 5. Maka 0 berada di depan dan 5 berada di paling belakang.

```

        System.out.println("Element Antrian "+q);

```

4. Mencetak isi Queue, saat objek q digabungkan dengan string, java memanggil method toString() bawaan dari AbstractCollection.

```

        System.out.println("Element Antrian "+q);
        int hapus_2017 = q.remove();
        System.out.println("Hapus Elemen = " + hapus_2017);
        System.out.println(q);

```

5. Operasi Dequeue `q.remove()` mengambil dan menghapus permanen elemen paling depan (0).

```
int depan_2017 = q.peek();
System.out.println("Kepala Antrian = " + depan_2017);
```

6. Operasi peek `q.peek()` mengembalikan nilai elemen paling depan tanpa menghapusnya. Maka yang akan dimunculkan adalah 1 karena 0 sudah dihapus.

```
int banyak_2017 = q.size();
System.out.println(" Size Antrian = "+banyak_2017);

}

}
```

7. Operasi ukuran `q.size()` mengembalikan jumlah elemen yang masih tersimpan di dalam queue.
8. Output

```
Element Antrian [0, 1, 2, 3, 4, 5]
Hapus Elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

- ReverseData_2511532017

1. Kita akan membuat class baru untuk ReverseData_2511532017

```
package pekan4_2511532017;

import java.util.*;

public class ReverseData_2511532017 {

    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<Integer>();
        q.add(1);
        q.add(2);
        q.add(3);
        System.out.println("Sebelum reverse" +q);
    }
}
```

2. `Queue<Integer> q = new LinkedList<>()` membuat antrian bertipe integer, `q.add(1)` menambahkan elemen 1 dengan 1 paling depan dan 3 paling akhir. Lalu dicetak sebagai penanda urutan sebelum di reverse.

```
Stack<Integer> s = new Stack<Integer>();
```

3. Membuat objek stack bertipw Integer. Dengan mengikuti prinsip LIFO maka elemen yang terakhir masuk akan pertama kali keluar.

```
while (!q.isEmpty()) {  
    s.push(q.remove());  
}
```

4. Looping transfer queue ke stack dimana queue mengeluarkan elemen dari depan (1-2-3) lalu stack menumpuknya dengan elemen yang pertama kali masuk akan berada di paling bawah (3-2-1).

```
while (!s.isEmpty()) {  
    q.add(s.pop());  
}  
System.out.println("sesudah di reverse= " + q);  
}  
}
```

5. Looping transfer stack ke queue, stack mengeluarkan dari atas dan queue memasukkan ke belakan secara berurutan dan hasil akhir 3,2,1
6. Output

```
Sebelum reverse[1, 2, 3]  
sesudah di reverse= [3, 2, 1]
```

- AntrianLoket_2511532017

1. Kita akan membuat kelas dengan nama AntrianLoket_2511532017

```
package pekan4_2511532017;  
import java.util.*;  
public class AntrianLoket_2511532017 {  
    int front_2017, rear_2017, size_2017;  
    int capacity_2017;  
    String array_2017[];
```

2. Deklrasi kelas dan atribut, front_2017 indeks elemen paling depan, rear_2017 indeks elemen paling belakang, size_2017 jumlah elemen yang sedang tersimpan dan array_2017[] tempat penyimpanan nama pelanggan

```

public AntrianLoket_2511532017 (int capacity_2017) {
    this.capacity_2017 = capacity_2017;
    front_2017 = this.size_2017=0;
    rear_2017 = capacity_2017-1;
    array_2017 = new String [this.capacity_2017];
}

```

3. this.capacity_2017 = capacity_2017 menyimpan kapasitas maksimal, front_2017 = this.size_2017 = 0 size jadi 0, lalu front juga jadi 0 dan rear_2017 = capacity_2017 - 1 dengan memulai rear di indeks terakhir, operasi (rear + 1) % capacity pada enqueue pertama akan menghasilkan 0, sehingga elemen pertama akan masuk ke indeks 0

```

//cek penuh atau belum
boolean isFull_2511532017() {
    return (this.size_2017 == this.capacity_2017);
}
//cek kosong atau enggak
boolean isEmpty_2511532017() {
    return (this.size_2017==0);
}

```

4. method pengecekan status penuh dan kosong, jika penuh maka true dan masih ada slot maka false. true jika kosong dan false jika ada elemen.

```

//masukkan antrian
void enqueue_2511532017 (String nama_2017) {
    if (isFull_2511532017 ()) {
        System.out.println("Antrian penuh");
        return;
    }

    this.rear_2017=(this.rear_2017+1) % this.capacity_2017;
    this.array_2017[this.rear_2017 ]= nama_2017;
    this.size_2017++;
    System.out.println( "Data berhasil ditambahkan ke antrian");
}

```

5. Cek penuh,jika size == capacity cetak pesan dan return. Jika rear = 4 dan capacity = 5 maka akan 0 lalu kembali ke awal array, masukkan nama_2017 ke array[rear] dan tambah size untuk mencatat elemen baru

```

//hapus antrian
String dequeue_2511532017 () {
    if (isEmpty_2511532017 ()) {
        System.out.println("Antrian kosong");
        return null;}
    String nama_2017= this.array_2017[this.front_2017];
    this.front_2017 = (this.front_2017+1) % this.capacity_2017;
    this.size_2017=this.size_2017-1;
    return nama_2017;
}

```

6. Cek kosong jika size==0 cetak pesan dan return null, ambil data dan simpan di array[front] ke variabel sementara, kurangi size dan kembalikan nama pelanggan tadi

```
String front_2511532017() {
    if (isEmpty_2511532017 ())return null;
    return this.array_2017[this.front_2017];
}
String rear_2511532017() {
    if (isEmpty_2511532017())
        return null;
    return this.array_2017[this.rear_2017];
}
```

7. Method front dan rear untuk menampilkan info siapa yang dilayani dan siapa yang paling dibelakang

```
//reverse antrian
public void reverseQueue_2511532017() {
    if(isEmpty_2511532017()) {
        System.out.println("antrian kosong"); return;
    }
    Stack<String> stack = new Stack <>();
    while (!isEmpty_2511532017()) {
        String nama_2017 = dequeue_2511532017();
        if(nama_2017 != null) {
            stack.push(nama_2017);
        }
    }
    while (!stack.isEmpty()) {
        enqueue_2511532017(stack.pop());
    }
}
```

8. Method reverse untuk membalikkan elemen queue menggunakan stack

```
//tampilkan antrian
void display_2511532017 () {
    if (this.size_2017 == 0) {
        System.out.println("\nAntrian Kosong\n");
        return;
    }
    System.out.println("isi antrian: ");
    for (int i=0; i<size_2017; i++) {
        int idx = (this.front_2017 +i)% this.capacity_2017;
        System.out.printf("\n   %d. %s", (i+1), array_2017[idx] );
    }
    System.out.println("\n");
}
}
```

9. Method untuk menampilkan antrian

- AntrianLoketDriver_2511532017

1. Kita akan membuat kelas driver dengan nama AntrianLoketDriver_2511532017

```

package pekan4_2511532017;
import java.util.Scanner;

public class AntrianLoketDriver_2511532017 {

    //method menu
    public static void tampilkanMenu_2511532017 () {
        System.out.println("\n=== Program Antrian Loret NIM: 2511532017 ===");
        System.out.println("1. Tambah Antrian");
        System.out.println("2. Hapus Antrian");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Reverse");
        System.out.println("5. keluar");
    }
}

```

2. Method tampilkan semua menu yang akan dipilih

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    AntrianLoket_2511532017 queue = new AntrianLoket_2511532017(5);
    int pilihan;
}

```

3. Method main() tempat awal eksekusi program. Membuat objek scanner untuk membaca input dari user, new AntrianLoket_2511532017(5) membuat objek queue dengan kapasitas maksimal 5 elemen dan int pilihan merupakan variabel untuk menyimpan nomor menu yang dipilih user

```

do {
    tampilkanMenu_2511532017();
    System.out.print("Pilih menu (1-5): ");
    try {
        pilihan = Integer.parseInt(sc.nextLine().trim());
    } catch (NumberFormatException e) {
        pilihan = -1;
        System.out.println("Input harus angka 1-5!");
        continue;
    }
}

```

4. Loop utama melakukan minimal satu kali sebelum mengecek kondisi (pilihan != 5).

```

do {
    tampilkanMenu_2511532017();
    System.out.print("Pilih menu (1-5): ");
    try {
        pilihan = Integer.parseInt(sc.nextLine().trim());
    } catch (NumberFormatException e) {
        pilihan = -1;
        System.out.println("Input harus angka 1-5!");
        continue;
    }
}

```

5. Membaca input dari pengguna dan mengubah String ke int dan menangkap error jika inputan berupa bukan angka.


```

switch (pilihan) {
    case 1:
        System.out.println("Masukkan nama pelanggan: ");
        String namaPelanggan_2017 = sc.nextLine();
        if(!namaPelanggan_2017.trim().isEmpty()) {
            queue.enqueue_2511532017(namaPelanggan_2017);
        } else {System.out.println("nama tidak boleh kosong");}

        break;
    case 2:
        String selesai_2017 = queue.dequeue_2511532017();
        if (selesai_2017 != null) {
            System.out.println(selesai_2017 + " telah dilayani ");
        }
        break;
    case 3:
        queue.display_2511532017();
        break;
    case 4:
        System.out.println("isi antrian");
        queue.reverseQueue_2511532017();
        queue.display_2511532017();
        break;
    default:
        System.out.println(" Pilihan tidak valid.");
}
} while (pilihan != 5);
sc.close();
}

```

6. Switch case digunakan sebagai penanganan menu
7. Output

```

=== Program Antrian Loret NIM: 2511532017 ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 1
Masukkan nama pelanggan:
dono
Data berhasil ditambahkan ke antrian

=== Program Antrian Loret NIM: 2511532017 ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 1
Masukkan nama pelanggan:
mono
Data berhasil ditambahkan ke antrian

=== Program Antrian Loret NIM: 2511532017 ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 3
isi antrian:

=== Program Antrian Loret NIM: 2511532017 ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 3
isi antrian:

    1. dono
    2. mono

=== Program Antrian Loret NIM: 2511532017 ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 1
Masukkan nama pelanggan:
yur
Data berhasil ditambahkan ke antrian

=== Program Antrian Loret NIM: 2511532017 ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 3
isi antrian:

    1. dono
    2. mono
    3. yur

```

=== Program Antrian Loker NIM: 2511532017 ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 4
isi antrian
Data berhasil ditambahkan ke antrian
Data berhasil ditambahkan ke antrian
Data berhasil ditambahkan ke antrian
isi antrian:

1. yur
2. mono
3. dono

=== Program Antrian Loker NIM: 2511532017 ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 4
isi antrian
Data berhasil ditambahkan ke antrian
Data berhasil ditambahkan ke antrian
Data berhasil ditambahkan ke antrian
isi antrian:

1. dono
2. mono
3. yur

=== Program Antrian Loker NIM: 2511532017 ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 2
dono telah dilayani

=== Program Antrian Loker NIM: 2511532017 ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5): 3
isi antrian:

1. mono
2. yur

=== Program Antrian Loker NIM: 2511532017 ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
Pilih menu (1-5):

BAB III

KESIMPULAN

Struktur data Queue berhasil diimplementasikan untuk mengatur elemen dengan prinsip FIFO, dimana elemen yang pertama kali masuk akan pertama kali keluar. Operasi dasarnya meliputi enqueue (tambah di rear), dequeue (hapus dari front), peek/front (intip elemen depan), isEmpty, dan isFull.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Queue Data Structure," *GeeksforGeeks*, 2024.
[Online]. Available: <https://www.geeksforgeeks.org/queue-data-structure/>.
[Accessed: Apr. 27, 2026].